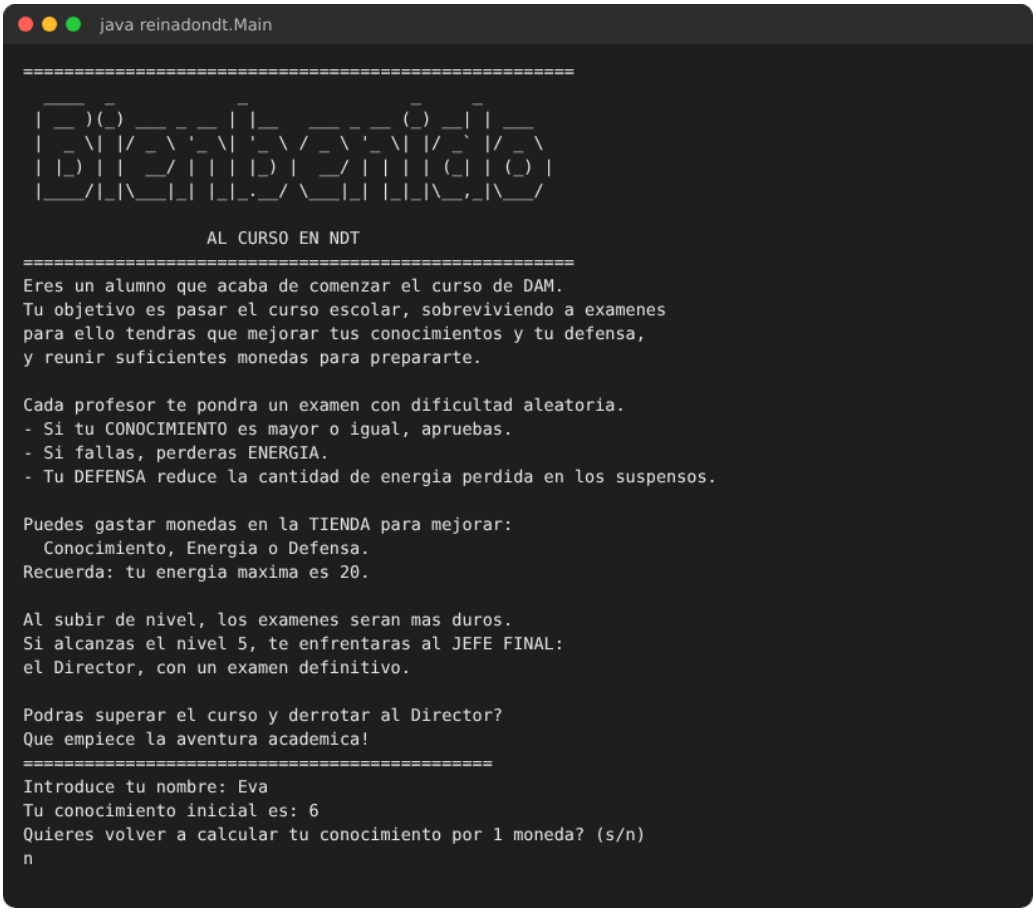


Reinado NDT

Juego de rol de texto en Java

Eva Gallardo Romero

18 de noviembre de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Inicio . . . 3
3.2 Menú y estadísticas . . . 3
3.3 Tienda . . . 3
3.4 Examen . . . 4

4 Requisitos del sistema **4**
4.1 Requisitos funcionales . . . 4
4.2 Requisitos no funcionales . . . 5

5 Documentación técnica **5**
5.1 Alumno.java . . . 5
5.2 Profesor.java . . . 6
5.3 Mejorable.java, TipoMejora.java y MejoraSimple.java . . . 6
5.4 Main.java . . . 7

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: ReinadoNDT.

La documentación del código generada con Javadoc está en evagr11.github.io/ReinadoNDT/javadoc.

1 Descripción general

Reinado NDT es un **juego de rol de texto** hecho en **Java** para la consola. Eres un alumno que acaba de empezar DAM y tiene que sobrevivir al curso: en lugar de dragones, los enemigos son los **exámenes de los profesores**.

- Cada examen tiene una **dificultad** aleatoria. Si tu **conocimiento** es mayor o igual, apruebas, ganas **monedas** y **subes de nivel**.
- Si suspendes pierdes **energía**, aunque la **defensa** reduce lo que pierdes. Si la energía llega a 0, pierdes el curso.
- Con las monedas se compra en la **tienda**: chuletas, horas de estudio, ayuda de un compañero, café, bebidas energéticas o apuntes organizados.
- Al llegar al **nivel 5** se puede retar al **Director**, el jefe final, con un examen de dificultad 20.

Fue una práctica de **Programación** en New Digital Talent (de ahí *NDT*) para aplicar la programación orientada a objetos en un programa de consola completo: clases con atributos privados, una **interfaz**, un **enum**, un `switch` con flechas y un método **recursivo** para la animación del examen.

2 Diagrama de clases

`Main` lleva la partida: crea al `Alumno`, un `Profesor` nuevo en cada examen y una `MejoraSimple` cada vez que se compra algo en la tienda. Las mejoras implementan la interfaz `Mejorable` y su tipo es un valor del enum `TipoMejora`.

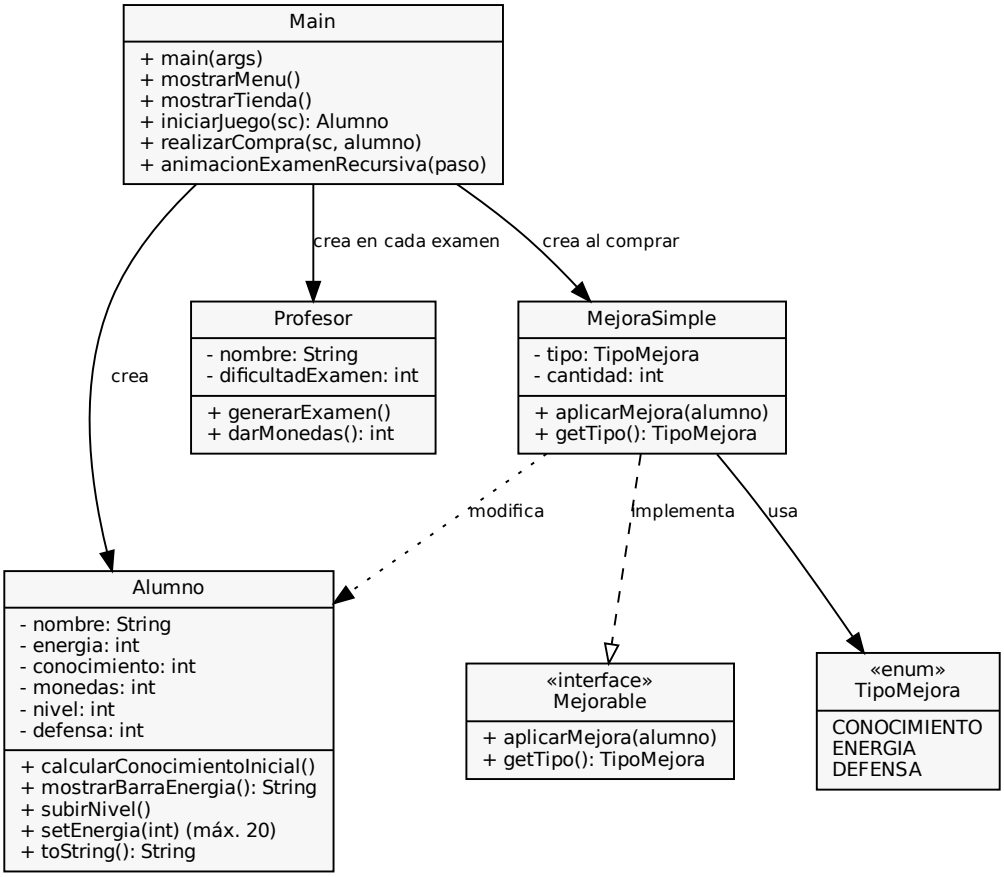


Figura 1: Diagrama de clases de Reinado NDT

3 Diseño

Toda la partida ocurre en la consola. El título se dibuja con **arte ASCII**, los menús van numerados y la energía se muestra como una **barra** de almohadillas. Las capturas son de partidas reales.

3.1 Inicio

Explica las reglas, pide el nombre y calcula el conocimiento inicial (de 1 a 10). Se puede volver a tirar pagando una moneda.

3.2 Menú y estadísticas

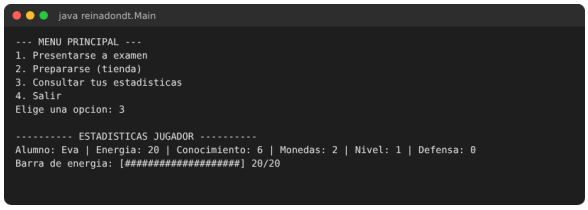
Cuatro opciones: presentarse a examen, ir a la tienda, consultar las estadísticas o salir. Las estadísticas muestran todos los atributos y la barra de energía.

3.3 Tienda

Seis productos con lo que mejoran y su precio. Si no hay monedas suficientes, avisa.

3.4 Examen

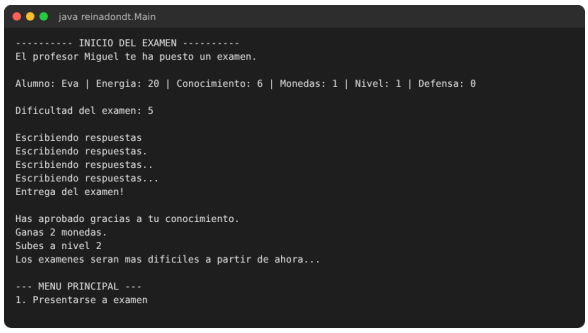
Sale el nombre del profesor, el estado del alumno y la dificultad. La entrega se anima con «Escribiendo respuestas» y puntos que aparecen uno a uno. Después se muestra si has aprobado (monedas y nivel nuevo) o si has suspendido (energía perdida y barra actualizada).



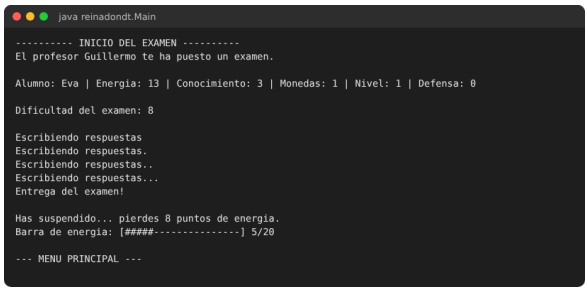
3.2. Estadísticas



3.3. Tienda



3.4. Examen aprobado



3.4. Examen suspendido

Figura 2: Capturas de dos partidas

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	El juego debe pedir el **nombre** y dar un **conocimiento inicial** aleatorio de 1 a 10, que se puede volver a calcular por 1 moneda.
RF02	El alumno debe empezar con **20 de energía**, 2 monedas y nivel 1. La energía no puede pasar de 20.
RF03	Cada examen lo debe poner un **profesor al azar** con una dificultad de 1 a 15.
RF04	Si el conocimiento es **mayor o igual** que la dificultad, el alumno aprueba, gana de 1 a 3 monedas y sube un nivel.
RF05	Si suspende, pierde tanta energía como la dificultad **menos su defensa**, y siempre al menos 1.
RF06	Si la energía llega a 0, el alumno **pierde el curso** y termina la partida.
RF07	La tienda debe vender seis mejoras de conocimiento, energía o defensa, comprobando que hay monedas.
RF08	Al llegar al **nivel 5** se debe ofrecer el examen final del **Director** (dificultad 20).
RF09	Las estadísticas deben mostrar todos los atributos y la **barra de energía**.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El programa debe funcionar en la **consola** con Java y sin librerías externas.
RNF02	Los atributos de las clases deben ser **privados**, con getters y setters.
RNF03	Las mejoras deben implementar una **interfaz** común para poder añadir tipos nuevos sin tocar la tienda.
RNF04	La animación del examen debe hacerse con un método **recursivo**.
RNF05	Todas las clases y métodos deben tener comentarios **Javadoc**.

5 Documentación técnica

Este anexo agrupa la documentación por clases, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `import`, los getters y setters sencillos y los comentarios Javadoc, que repiten la descripción.

5.1 Alumno.java

5.1.1 Ubicación

`reinadondt.Alumno`

Descripción general:
El jugador. Guarda el nombre, la energía, el conocimiento, las monedas, el nivel y la defensa.

5.1.2 Funciones

Alumno **Descripción:** Constructor: empieza con 20 de energía, 2 monedas y nivel 1.
Código comentado:

```
public Alumno(String nombre) {
    this.nombre = nombre;
    this.energia = 20;
    this.conocimiento = 0;
    this.monedas = 2;
    this.nivel = 1;
}
```

setEnergia **Descripción:** Cambia la energía sin dejar que pase de 20.
Código comentado:

```
public void setEnergia(int energia) {
    if (energia > 20) {
        this.energia = 20;          // la energía máxima es 20
    } else {
        this.energia = energia;
    }
}
```

calcularConocimientoInicial **Descripción:** Da un conocimiento aleatorio de 1 a 10.
Código comentado:

```
public void calcularConocimientoInicial() {
    int fuerza = (int)(Math.random() * 10) + 1;
    this.conocimiento = fuerza;
}
```

mostrarBarraEnergia **Descripción:** Dibuja la energía como una barra de 20 caracteres.
Código comentado:

```
public String mostrarBarraEnergia() {
    int maxEnergia = 20;
    int longitudBarra = 20;
    int relleno = (int) Math.round((double) energia / maxEnergia * longitudBarra);
    if (relleno > longitudBarra) relleno = longitudBarra;
    if (relleno < 0) relleno = 0;

    StringBuilder barra = new StringBuilder("[");
    for (int i = 0; i < longitudBarra; i++) {
        if (i < relleno) barra.append("#");      // parte llena
        else barra.append("-");                  // parte vacía
    }
    barra.append("] ").append(energia).append("/").append(maxEnergia);
    return barra.toString();
}
```

5.2 Profesor.java

5.2.1 Ubicación

`reinadondt.Profesor`

Descripción general:
Un profesor con nombre que pone exámenes y da monedas al aprobar.

5.2.2 Funciones

generarExamen y darMonedas **Descripción:** La dificultad es aleatoria de 1 a 15 y la recompensa de 1 a 3 monedas.
Código comentado:

```
public void generarExamen() {
    this.dificultadExamen = (int)(Math.random() * 15) + 1;
}

public int darMonedas() {
    return (int)(Math.random() * 3) + 1;
}
```

5.3 Mejorable.java, TipoMejora.java y MejoraSimple.java

5.3.1 Ubicación

`reinadondt`

Descripción general:
`Mejorable` es la interfaz de las mejoras de la tienda, `TipoMejora` el enum con los tres atributos que se pueden mejorar y `MejoraSimple` la mejora que suma una cantidad a uno de ellos.

5.3.2 Funciones

Mejorable **Descripción:** Interfaz con los dos métodos que debe tener cualquier mejora.
Código comentado:

```
public interface Mejorable {
    void aplicarMejora(Alumno alumno);
    TipoMejora getTipo();
}

public enum TipoMejora {
```

```
    CONOCIMIENTO,
    ENERGIA,
    DEFENSA
}
```

aplicarMejora **Descripción:** Suma la cantidad al atributo que indica el tipo, con un `switch` de flechas.

Código comentado:

```
@Override
public void aplicarMejora(Alumno alumno) {
    switch (tipo) {
        case CONOCIMIENTO -> alumno.setConocimiento(alumno.getConocimiento() +
            cantidad);
        case ENERGIA -> alumno.setEnergia(alumno.getEnergia() + cantidad);   // máx.
            20
        case DEFENSA -> alumno.setDefensa(alumno.getDefensa() + cantidad);
    }
}
```

5.4 Main.java

5.4.1 Ubicación

`reinadondt.Main`

Descripción general:
El programa principal: la introducción, el menú, los exámenes, la tienda y el jefe final.

5.4.2 Funciones

iniciarJuego **Descripción:** Muestra las reglas, crea el alumno y deja repetir el conocimiento inicial por una moneda.

Código comentado:

```
System.out.print("Introduce tu nombre: ");
String nombreAlumno = sc.nextLine();
Alumno alumno = new Alumno(nombreAlumno);
alumno.calcularConocimientoInicial();
System.out.println("Tu conocimiento inicial es: " + alumno.getConocimiento());

System.out.println("Quieres volver a calcular tu conocimiento por 1 moneda? (s/n)");
String opcion = sc.nextLine();
if (opcion.equalsIgnoreCase("s") && alumno.getMonedas() > 0) {
    alumno.setMonedas(alumno.getMonedas() - 1);
    alumno.calcularConocimientoInicial();
    System.out.println("Nuevo conocimiento: " + alumno.getConocimiento());
}
return alumno;
```

main (examen) **Descripción:** Fragmento del `case 1`: elige un profesor al azar, compara el conocimiento con la dificultad y aplica el resultado.

Código comentado:

```
case 1: // Examen
    int random = (int)(Math.random() * nombresProfesores.length);
    Profesor profesor = new Profesor(nombresProfesores[random]);
    profesor.generarExamen();
    animacionExamenRecursiva(0);

    int ataqueEnemigo = profesor.getDificultadExamen();
    if (alumno.getConocimiento() >= ataqueEnemigo) {
        int monedasGanadas = profesor.darMonedas();
        alumno.setMonedas(alumno.getMonedas() + monedasGanadas);
        alumno.subirNivel();
```

```
        // a partir del nivel 5 se ofrece el examen del Director (dificultad 20)
    } else {
        int dañoBase = ataqueEnemigo;
        int reduccion = alumno.getDefensa();
        int dañoFinal = dañoBase - reduccion;
        if (dañoFinal < 1) dañoFinal = 1;           // siempre se pierde algo
        alumno.setEnergia(alumno.getEnergia() - dañoFinal);
        System.out.println("Barra de energia: " + alumno.mostrarBarraEnergia());
        if (alumno.getEnergia() <= 0) {
            System.out.println("Tu energia ha llegado a 0. Has perdido el curso!");
            jugando = false;
        }
    }
    break;
```

animacionExamenRecursiva **Descripción:** Escribe «Escribiendo respuestas» con 0, 1, 2 y 3 puntos, llamándose a sí misma con el paso siguiente hasta entregar el examen.

Código comentado:

```
public static void animacionExamenRecursiva(int paso) {
    if (paso > 3) {                          // caso base
        System.out.println("Entrega del examen!\n");
        return;
    }
    StringBuilder animacion = new StringBuilder("Escribiendo respuestas");
    for (int j = 0; j < paso; j++) animacion.append(".");
    for (int j = paso; j < 3; j++) animacion.append(" ");
    System.out.println(animacion.toString());
    try {
        Thread.sleep(500);                   // medio segundo entre pasos
    } catch (InterruptedException e) {
        System.out.println("Error en la animacion.");
    }
    animacionExamenRecursiva(paso + 1);      // llamada recursiva
}
```

realizarCompra **Descripción:** Muestra la tienda y aplica la mejora elegida si hay monedas. Se muestran dos de los seis productos; el resto sigue el mismo patrón.

Código comentado:

```
public static void realizarCompra(Scanner sc, Alumno alumno) {
    mostrarTienda();
    int compra = sc.nextInt();
    switch (compra) {
        case 1: // Chuleta
            if (alumno.getMonedas() >= 2) {
                alumno.setMonedas(alumno.getMonedas() - 2);
                new MejoraSimple(TipoMejora.CONOCIMIENTO, 3).aplicarMejora(alumno);
                System.out.println("\nHas conseguido una chuleta (+3 conocimiento).");
            } else System.out.println("\nNo tienes suficientes monedas.");
            break;
        case 6: // Apuntes organizados
            if (alumno.getMonedas() >= 2) {
                alumno.setMonedas(alumno.getMonedas() - 2);
                new MejoraSimple(TipoMejora.DEFENSA, 2).aplicarMejora(alumno);
                System.out.println("\nHas conseguido apuntes organizados (+2 defensa).
                    ");
            } else System.out.println("\nNo tienes suficientes monedas.");
            break;
        default:
            System.out.println("\nOpcion no valida.");
    }
}
```